



FIREBRAND

LYCEUM EDUCATION GROUP

Technology Stacks and APIs

How the front end, back end, database and API fit together, then fetch brings data to life.

What You'll Be Able to Do by This Evening

By the end of today you can explain a stack and pull live data onto a page.

1

Explain the stack: describe what the front end, back end, database and API each do

2

Read a data flow: trace a request from button click to displayed result

3

Use fetch: request data from a real public API using JavaScript

4

Handle a response: read JSON data using .then and display it with the DOM

5

Communicate clearly: explain what an API call does to both technical and non-technical listeners

One Button Click, Four Systems

A patient taps "Check my appointment" on an NHS app, and something happens.

1

CONTEXT

The patient sees a spinner for half a second, then their appointment time appears on screen.

2

CHALLENGE

That half second involves several separate pieces of software, running in different places, talking to each other.

3

TASK

In pairs, guess how many separate "systems" might be involved in fetching that one appointment.

A Technology Stack, Defined

A stack is the set of layers a piece of software is built from, working together.

Front end

what the user sees and interacts with in the browser

Back end

the server-side logic that processes requests and applies rules

Database

where data is stored and retrieved from long term

API

the agreed contract that lets these layers talk to each other

The Stack, Standing Together

Four layers, each doing a distinct job, connected by requests and responses.

Front end

the browser page the user sees and clicks on

API

the contract defining what can be asked for and what comes back

Back end

server logic that processes the request and applies rules

Database

long-term storage the back end reads from and writes to

Spot the Layer

Sort five everyday actions into the stack layer responsible for them.

- 1 Read the five actions on the handout or shared doc
- 2 Write front end, back end, database or API next to each
- 3 Compare answers with a partner before checking together



TIMEBOX

8 minutes total



OUTPUT

Five actions labelled with the correct stack layer, with reasoning noted

The Piece That Ties It Together

“

An API is a contract, not a conversation.

Framing for today

What an API Actually Provides

An API defines what you can ask for, how to ask, and what shape the answer takes.

01

Endpoint: a specific web address representing one type of request

02

Request: what your code sends, sometimes with extra details attached

03

Response: the data sent back, usually in a structured text format called JSON

04

Rules: what is and isn't allowed, and what happens if the request is invalid

From Object to JSON and Back

A JavaScript object, built in code

- `const patient` equals an object with name Amara and status Confirmed

The same data as JSON, arriving from an API

- Braces containing `"name": "Amara", "status": "Confirmed"`

Same Shape, Three Sectors



Healthcare API

Returns appointment slots and clinic availability as structured data for a booking page.

Banking API

Returns account balance and recent transactions for a mobile banking app.

Retail API

Returns live stock levels and delivery estimates for a product page.

Logistics API

Returns a parcel's current location and estimated delivery time for a tracking page.

Where This Sits in the Bigger Picture

Your organisation's own version of the lifecycle decides who builds which layer.

Some teams own the whole stack: front end, back end and database, end to end

Some teams only own the front end, consuming APIs built and owned elsewhere

Some organisations buy in a third-party API rather than building their own

Your specific role on this programme is most likely front-end focused for now

Naming the Layers, Fast

Rapid-fire round naming which stack layer three real features belong to.

- 1 Read each feature description on the shared slide
- 2 Say out loud which layer is mainly responsible
- 3 Discuss any disagreement briefly with your group

TIMEBOX

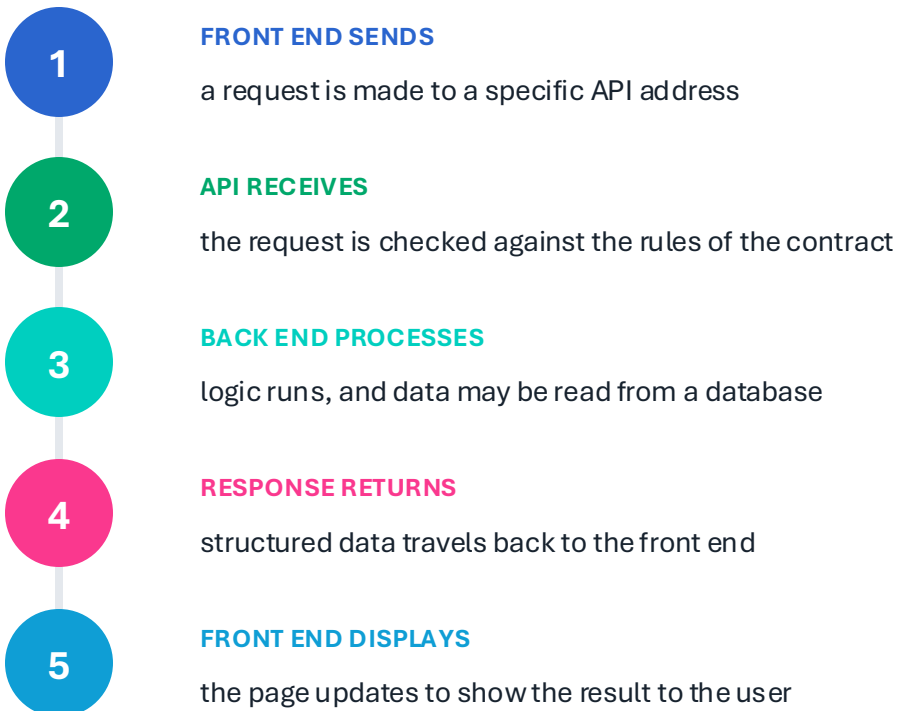
6 minutes total

OUTPUT

Three features confidently matched to a stack layer, with reasoning given

The Request and Response Journey

Every API call follows the same round trip, whichever sector it serves.



Predict the Flow

Match five app behaviours to the correct stage in the request and response journey.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

Five behaviours correctly matched to a stage in the request and response flow

BRIEFING STEPS

- 01** Read the five behaviours on the shared handout
- 02** Match each to a stage from the previous flow diagram
- 03** Check your answers against a partner's before we reveal

 **8 minutes**
total

Two Kinds of Database, Same Job

Relational database



Data stored in tables with fixed columns, like rows in a spreadsheet

Non-relational database



Data stored more flexibly, often as documents similar to JSON objects

Choosing a Database, By Sector

The right choice depends on how uniform the data is, not on which is "better".



NHS patient records

Highly structured, consistent fields across every patient, often relational



Retail product catalogue

Widely varying product attributes across categories, often non-relational



Banking transaction history

Strict structure and auditability required, often relational



Logistics tracking events

Varied event types per parcel journey, often non-relational

Only What's Shared

An API can hide far more than it reveals.

A security-minded framing

Design a Response, On Paper

You are deciding what one API endpoint should be allowed to return.

1

CONTEXT

A hospital appointment app needs to show patients their next appointment time and location.

2

CHALLENGE

The full patient record in the database includes far more than that, including clinical notes.

3

TASK

Decide, as a group, exactly which fields this specific API endpoint should return.

Sort the Fields

Sort ten database fields into "safe to return" and "should stay hidden".

- Instructions Read the ten fields listed on the shared handout Sort each into safe to return or should stay hidden Agree your final list with your group
- Timebox 10 minutes total
- Output Two labelled lists covering all ten given fields, with brief reasoning

Morning Recap, In Your Own Words

- 1 Which two layers of the stack have you personally already built code for on this programme?
- 2 What's one thing an API is responsible for that the front end never sees directly?
- 3 If a friend outside tech asked "what's an API", what would you say in one sentence?
- 4 Discuss your answers briefly with the person next to you.



FIREBRAND
LYCEUM EDUCATION GROUP

Lunch

Back at the time confirmed by your trainer, ready to write your first fetch request.



FIREBRAND

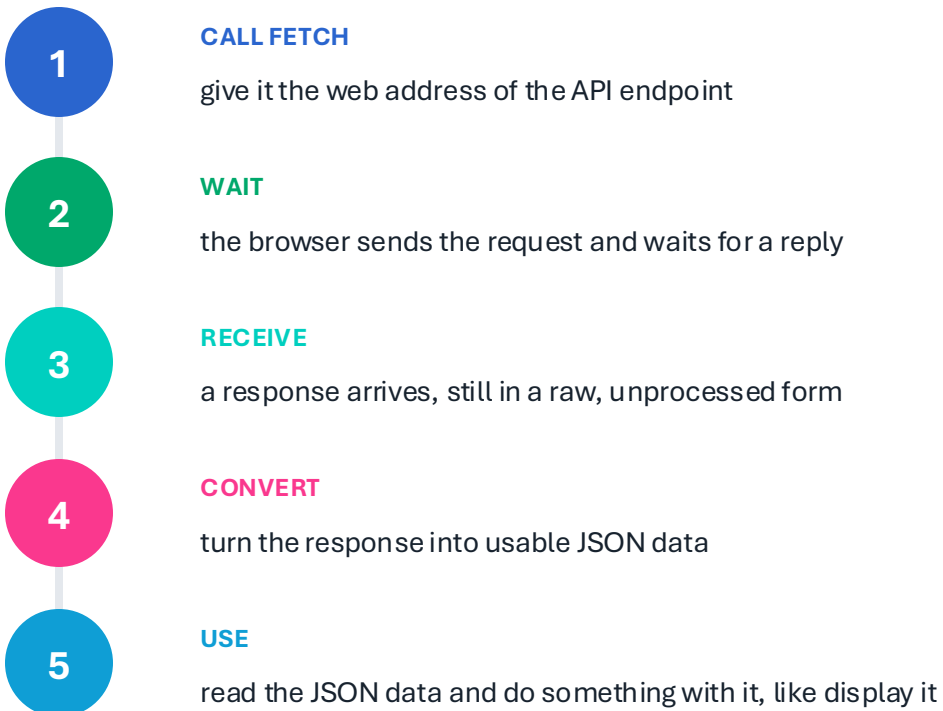
LYCEUM EDUCATION GROUP

This Afternoon: Fetch and Live Data

Requesting real data from a public API and displaying it using skills you already have.

What Fetch Actually Does

Fetch sends a request, waits, and hands back a response your code can read.



Your First Fetch, Line by Line

JavaScript

```
1 fetch("https://catfact.ninja/fact")
2   .then(function(response) {
3     return response.json();
4   })
5   .then(function(data) {
6     console.log(data);
7   });
```

1

fetch call

sends a request to the given web address

2

then block

runs once the response has arrived

3

response.json

converts the raw response into usable data

4

second then

runs once the JSON conversion is complete

Predict the Console Output

```
first-fetch.js

1 fetch("https://catfact.ninja/fact")
2   .then(function(response) {
3     return response.json();
4   })
5   .then(function(data) {
6     console.log(data);
7   });
```

CONSOLE OUTPUT

Object with a fact property containing a random cat fact, and a length property

Run Your First Fetch

Open first-fetch-starter.js and run it to see live data arrive.

- 1 Open first-fetch-starter.js in your editor
- 2 Run the file and open the browser console
- 3 Note what property names appear in the object
- 4 Run it a second time and compare the fact text

TIMEBOX

10 minutes total

OUTPUT

A printed object in the console containing a random cat fact, seen at least twice

Reaching Inside the Response

read-the-fact.js

```
1 fetch("https://catfact.ninja/fact")
2   .then(function(response) {
3     return response.json();
4   })
5   .then(function(data) {
6     console.log(data.fact);
7   });
```

VARIABLE	VALUE
data	an object with a fact property and a length property
data.fact	the random cat fact text itself
console.log(data.fact)	prints just the fact text, not the whole object

Read Just the Fact

Change the previous file to print only the fact text, not the whole object.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

Console showing only the cat fact text, confirmed across three runs

BRIEFING STEPS

- 01** Open first-fetch-starter.js again, or duplicate it
- 02** Change `console.log(data)` to `console.log(data.fact)`
- 03** Run it and confirm only text appears in the console
- 04** Run it twice more and note the fact changes each time

 **8 minutes**
total

From Console to Page

“

Printing to the console is the last step before showing something real to a user.

Framing for the DOM connection

Displaying a Fetched Fact on the Page

JavaScript

```
1 const factBox = document.querySelector("#fact-display");
2 fetch("https://catfact.ninja/fact")
3   .then(function(response) {
4     return response.json();
5   })
6   .then(function(data) {
7     factBox.textContent = data.fact;
8   });
```

1

querySelector

finds the element already on the page to update

2

fetch call

requests the data exactly as before

3

data.fact

reads the property off the response, as just practised

4

textContent

writes the fact into the page, replacing placeholder text

Build the Fact Display Page

Open fact-display-starter.js and wire it to show a live fact on the page.

- Instructions Open fact-display.html and fact-display-starter.js together Check the paragraph id matches the querySelector string Run the page and confirm the fact replaces the placeholder text Refresh the page and confirm the fact changes each time
- Timebox 12 minutes total
- Output A working page showing a different live cat fact on each refresh

Two Ways to Explain the Same Line

Explain Your Fact Display, Two Ways

Write two short explanations of your fact display page for two different audiences.

- 1 Write one technical sentence, using `fetch`, `.then` and `JSON`
- 2 Write one non-technical sentence, with no code words at all
- 3 Read both aloud to a partner
- 4 Ask your partner which version they understood faster



TIMEBOX

10 minutes total



OUTPUT

**Two written sentences explaining
the same code to two different
audiences**

A Question About Data From the Internet

Data from an API is a guest in your code, not a member of the family.

A caution worth remembering

Wiring Fetch to a Button Click

Instead of fetching automatically, wait for the user to ask for a new fact.

1

SELECT THE BUTTON

using `querySelector`, exactly as on Day 11

2

ADD A LISTENER

`addEventListener` waits for a click

3

RUN THE FETCH

inside the listener function, only when clicked

4

UPDATE THE PAGE

exactly the same `textContent` line as before

New Fact on Click

JavaScript

```
1 const newFactButton = document.querySelector("#new-fact-button");
2 const factBox = document.querySelector("#fact-display");
3 newFactButton.addEventListener("click", function() {
4   fetch("https://catfact.ninja/fact")
5     .then(function(response) {
6       return response.json();
7     })
8     .then(function(data) {
9       factBox.textContent = data.fact;
10    });
11 });
```

1

button select

finds the button element on the page

2

addEventListener

waits for a click, exactly as on Day 11

3

fetch inside the listener

only runs once the button is clicked

4

textContent update

displays the new fact, replacing the previous one

Build the New Fact Button

Open new-fact-starter.js and wire the button to fetch a new fact on click.

- 1 Open new-fact.html and new-fact-starter.js together
- 2 Check both element ids match the querySelector strings
- 3 Click the button several times and watch the fact change
- 4 Try clicking quickly twice in a row and observe what happens

TIMEBOX

12 minutes total

OUTPUT

A working button that displays a new random fact on every click

Two Endpoints, Compared

Cat facts endpoint

- <https://catfact.ninja/fact>, returns an object with a fact and a length

Dog image endpoint

- <https://dog.ceo/api/breeds/image/random>, returns an object with a message and a status

Displaying a Fetched Image

JavaScript

```
1 const dogButton = document.querySelector("#new-dog-button");
2 const dogImage = document.querySelector("#dog-photo");
3 dogButton.addEventListener("click", function() {
4   fetch("https://dog.ceo/api/breeds/image/random")
5     .then(function(response) {
6       return response.json();
7     })
8     .then(function(data) {
9       dogImage.src = data.message;
10    });
11 });
```

1

fetch call

requests a random dog image from a different public API

2

data.message

this API names its main property message, not fact

3

img element

an image tag selected the same way as any other element

4

src property

setting an image's source shows a new picture on the page

Build the Dog Photo Button

Open dog-photo-starter.js and wire the button to show a new photo on click.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

A working button that displays a new random dog photo on every click

BRIEFING STEPS

- 01** Open dog-photo.html and dog-photo-starter.js together
- 02** Check the button and image ids match the querySelector strings
- 03** Click the button and confirm a new dog photo appears each time
- 04** Compare this code with new-fact-starter.js from earlier

 **12 minutes**
total

What Have You Actually Built Today

0

Clear

You can explain what a stack layer is responsible for Front end, back end, database and API each have a distinct, definable job.

1

Clear

You can write a basic fetch call with .then The two-step then chain, converting a response and then using the data, is familiar now.

2

Clear

You can combine fetch with the DOM and events Wiring a button click to a live data request reuses Day 10 and 11 skills directly.

3

Flag

Handling something going wrong with a request This is a real, useful next step, but deliberately not covered on this programme yet.

4

The Building Blocks of the Main Build

The afternoon build combines four elements you've already practised today.

A button

selected with `querySelector` and listened to with `addEventListener`

A fetch call

requesting data from a public API

A then chain

converting the response and reading the data

A `textContent` update

displaying the result on the page

Advice Generator, Starter Structure

The starter file has the button and element already selected, ready for you to complete.



advice-generator-starter.js

```
1 const newAdviceButton = document.querySelector("#new-advice-button");  
2 const adviceBox = document.querySelector("#advice-display");  
3 newAdviceButton.addEventListener("click", function() {  
4 });
```

The Nested Property, Explained

```
advice-shape.js

1 fetch("https://api.advice-slip.com/advice")
2   .then(function(response) {
3     return response.json();
4   })
5   .then(function(data) {
6     console.log(data);
7   });
```

VARIABLE	VALUE
data	an object containing one property called slip
data.slip	an object containing id and advice properties
data.slip.advice	the actual piece of advice text

Main Build, The Advice Generator

Complete the advice generator so clicking the button shows new advice each time.

- Instructions Open `advice-generator-starter.js` and `advice-generator.html` together Add a fetch call for the advice API inside the click listener Use `data.slip.advice` to read the text, then set it with `textContent` Click the button several times and confirm new advice appears
- Timebox 18 minutes total
- Output A working page showing a new piece of advice on every button click

Compare Your Build With a Partner

Swap screens with a partner and compare how you each wrote the click listener.

- 1 Show your partner your completed advice-generator-starter.js
- 2 Find one line that is written identically to theirs
- 3 Find one line that is written slightly differently
- 4 Discuss why both versions still work correctly



TIMEBOX

8 minutes total



OUTPUT

**One identified matching line and
one identified differing line, with
reasoning**

One More Sector, One More Wrapper

The same fetch-and-display pattern works for far more than facts and advice.

1

CONTEXT

A logistics company wants a simple page showing a live estimated delivery time for a parcel.

2

CHALLENGE

The underlying pattern, button, fetch, then chain, `textContent`, is identical to what you just built.

3

TASK

Imagine the JSON shape a delivery API might return, and sketch the property path you'd need.

Sketch a Property Path

Write the dot notation path you'd use for three different imagined API responses.

- 1 Read the three imagined JSON shapes on the shared handout
- 2 Write the exact dot notation path to reach the useful value in each
- 3 Compare your three answers with a partner

TIMEBOX

8 minutes total

OUTPUT

**Three correctly written dot notation paths
matching the given JSON shapes**

The Stack, One Last Time

Everything today connects back to this same four-layer picture from this morning.

Front end

the button, the page, and the `textContent` update you wrote

API

the contract defining the address and the JSON shape you read

Back end

the logic running somewhere else, generating the response

Database

wherever the underlying fact, advice or image was stored

Exit Ticket

Answer four short questions summarising today's stack and fetch learning.

CASE FILE — ACTIVITY BRIEFING

OBJECTIVE

Four short written answers reflecting today's key stack and fetch ideas

BRIEFING STEPS

- 01** Name the four layers of a technology stack
- 02** Explain in one sentence what `.then` is waiting for
- 03** Write the dot notation path for a nested JSON property you used today
- 04** Name one thing you'd want to learn next about working with APIs

 **10 minutes**
total

Closing: Stacks, APIs and Fetch

Today you connected everything you've built so far to real, live data from the internet.

- ✓ A stack is four connected layers, each with a distinct, definable job
- ✓ An API is a contract, defining exactly what can be asked for and returned
- ✓ Fetch and .then bring live JSON data into code you already know how to write
- ✓ The same pattern works identically across healthcare, finance, retail and beyond